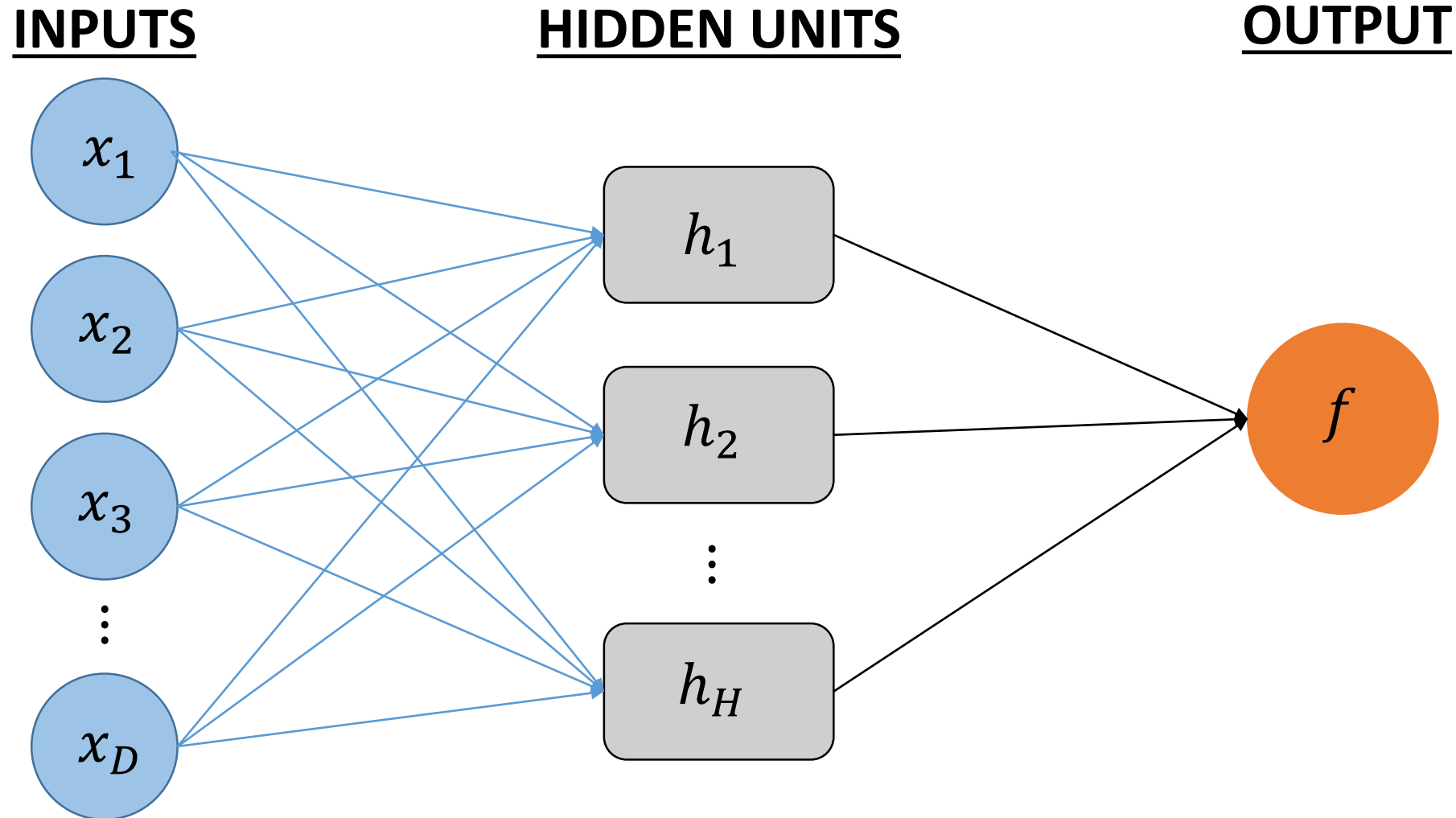


INFSCI 2595: Machine Learning

Week 14: Neural Networks

Spring 2024

Quick review of the single layer feed forward neural network for regression



We stepped through the calculations to go from input to response

Hidden layer – consists of two operations

- Linear combination of the inputs – essentially the “linear predictors”:

$$\eta_{n,k} = \sum_{d=0}^D (\beta_{d,k} x_{n,d})$$

- Non-linear transformation through a function $g(\cdot)$:

$$h_{n,k} = g(\eta_{n,k})$$

We stepped through the calculations to go from input to response

Hidden layer – consists of two operations

- Linear combination of the inputs – essentially the “linear predictors”:

$$\eta_{n,k} = \sum_{d=0}^D (\beta_{d,k} x_{n,d})$$

n -th observation
 k -th hidden unit
 d -th input
 $x_{n,0} = 1!$

- Non-linear transformation through a function $g(\cdot)$:

$$h_{n,k} = g(\eta_{n,k})$$

We stepped through the calculations to go from input to response

Output layer – combination of the hidden units.

Essentially a linear model of the unobserved hidden unit values!

$$f_n = \sum_{k=0}^H (\alpha_k h_{n,k})$$

We stepped through the calculations to go from input to response

Output layer – combination of the hidden units.

Essentially a linear model of the unobserved hidden unit values!

$$f_n = \sum_{k=0}^H (\alpha_k h_{n,k})$$

n -th observation
 k -th hidden unit
 $h_{n,0} = 1!$

We discussed how a neural network is essentially a series of matrix operations!

- Bind all hidden unit parameters into a matrix: $\mathbf{B} = [\boldsymbol{\beta}_1 \quad \boldsymbol{\beta}_2 \quad \cdots \quad \boldsymbol{\beta}_H]$
- All hidden unit “linear predictors” are equal to:

$$\mathbf{A} = \mathbf{XB}$$

- All hidden unit non-linear values are then:

$$\mathbf{H} = g(\mathbf{A}) = g(\mathbf{XB})$$

- Neural network response calculated with output layer intercept (bias) α_0 and column vector of “weights” $\boldsymbol{\alpha}$:

$$\mathbf{f} = \alpha_0 + \mathbf{H}\boldsymbol{\alpha} = \alpha_0 + [g(\mathbf{XB})]\boldsymbol{\alpha}$$

We discussed that the hidden units are essentially a non-linear basis adapted to the training data

- Analogous to the basis design matrix, Φ , we used for linear basis function models.

$$\mathbf{f} = \alpha_0 + \mathbf{H}\alpha \rightarrow \alpha_0 + \Phi\alpha$$

- However, rather than specifying the basis up front, neural networks learn the features from the data.

We introduced the learning problem

- For regression, the conventional **loss** function is to minimize the sum of squared errors (residuals):

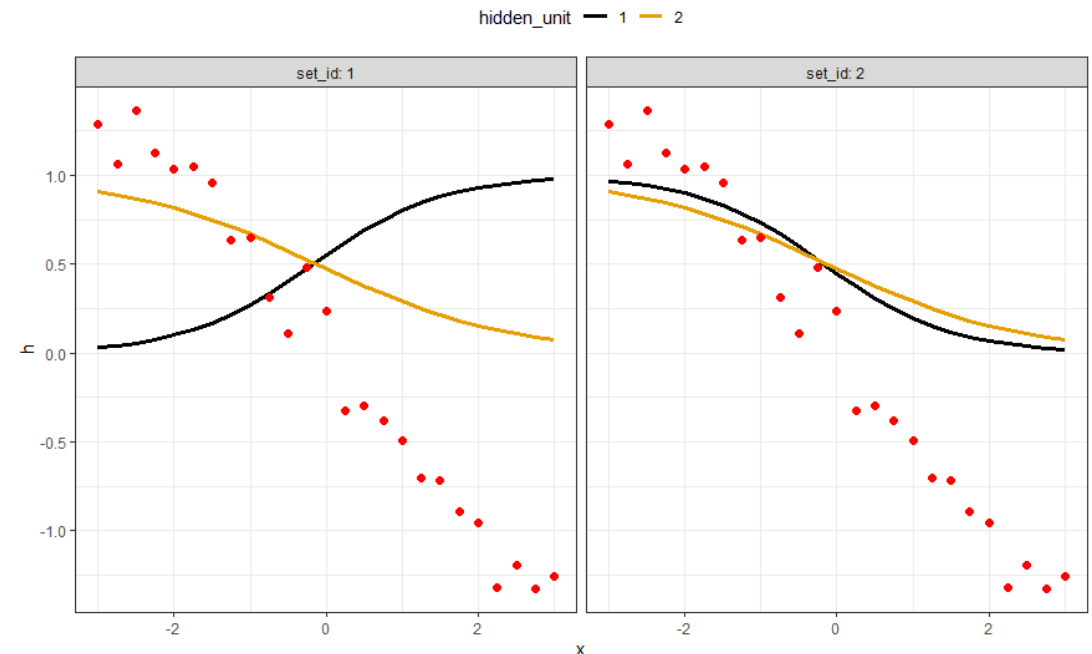
$$SSE = \sum_{n=1}^N ((y_n - f_n)^2)$$

- The optimization problem is to find the parameters that minimize the *SSE*:

$$\arg \min_{\beta_1, \beta_2, \dots, \beta_H, \alpha_0, \alpha} SSE$$

We qualitatively discussed the challenges of training neural networks with a simple example

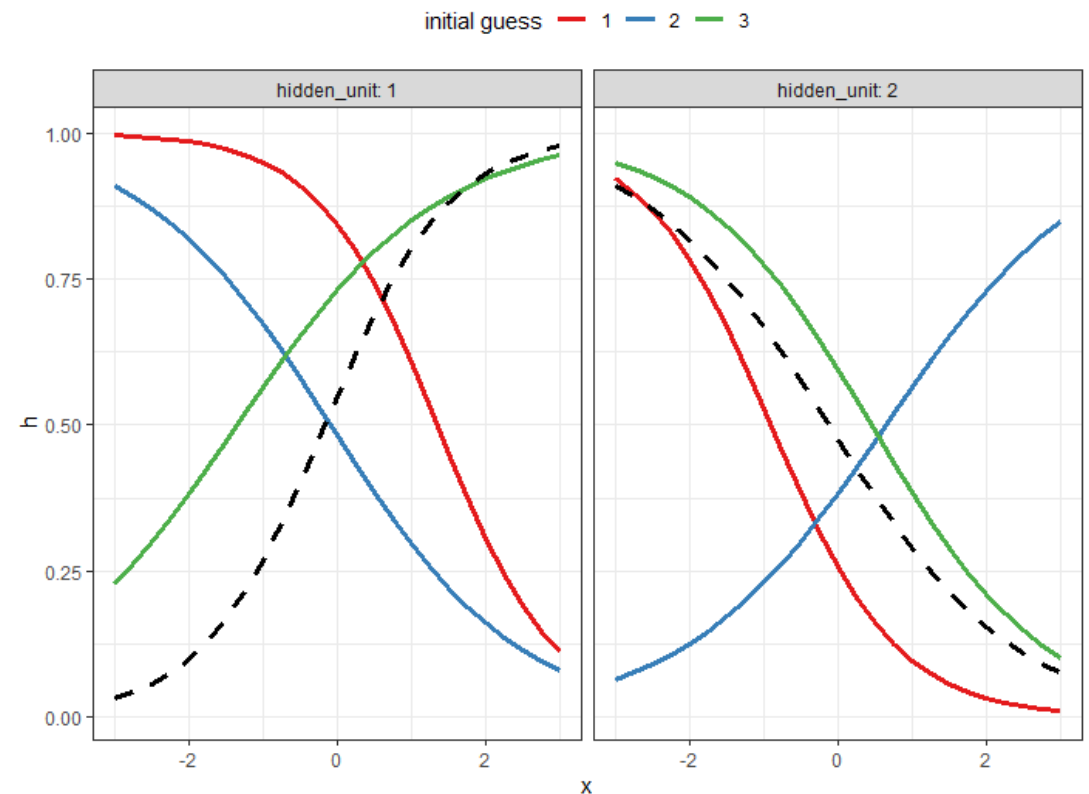
- We specified a simple toy problem.
- Single input, $D = 1$, and two hidden units, $H = 2$.
- Total of 7 weights and biases in the model.



Can we find the parameters that generated the data?

We qualitatively discussed the challenges of training neural networks with a simple example

- We specified a simple toy problem.
- Single input, $D = 1$, and two hidden units, $H = 2$.
- Total of 7 weights and biases in the model.



Initial guess impacts the optimal parameter estimates!
Neural network objective surface is multi-modal!

However, we did not go through what happens behind the scenes

- I said that you have all tools necessary to fit a neural network from scratch...
- Rather than calling `neuralnet`, or another library, we can program the objective function ourselves.
- Let's see how to do that with `optim()` in R.

With `optim()` all we need to do is setup the model...

- It handles everything else!
- But, what does “everything else” mean?

With `optim()` all we need to do is setup the model...

- It handles everything else!
- But, what does “everything else” mean?
- Everything else → iterate from an initial guess to the optimized estimates!

Iterative optimization algorithm nomenclature

- Group all parameters together: $\boldsymbol{\theta} = \left\{ \left\{ \{\beta_{dk}\}_{d=0}^D \right\}_{k=1}^H, \{\alpha_k\}_{k=0}^H \right\}$
- Or in vector notation: $\boldsymbol{\theta} = \{\boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \dots, \boldsymbol{\beta}_k, \dots, \boldsymbol{\beta}_H, \alpha_0, \boldsymbol{\alpha}\}$
- Define an initial guess for the parameters: $\boldsymbol{\theta}_{t=0}$
 - Subscript t will be used to denote the iteration
 - $t = 0$ corresponds to the starting or initial guess to the algorithm

Calculate the gradient to find the direction to move to minimize the SSE

- The gradient vector, $\mathbf{g}_t$, is the gradient at the iteration t .
- Each element in the gradient is the partial first derivative of the SSE with respect to the parameters:

$$\mathbf{g}_t = \left\{ \frac{\partial}{\partial \theta_{t,j}} (SSE_t) \right\}_{j=1}^J$$

Calculate the gradient to find the direction to move to minimize the SSE

- The gradient vector, $\mathbf{g}_t$, is the gradient at the iteration t .
- Each element in the gradient is the partial first derivative of the SSE with respect to the parameters:

$$\mathbf{g}_t = \left\{ \frac{\partial}{\partial \theta_{t,j}} (SSE_t) \right\}_{j=1}^J$$

t -th iteration, parameter j

Calculate the gradient to find the direction to move to minimize the SSE

- The gradient vector, $\mathbf{g}_t$, is the gradient at the iteration t .
- Each element in the gradient is the partial first derivative of the SSE with respect to the parameters:

$$\mathbf{g}_t = \left\{ \frac{\partial}{\partial \theta_{t,j}} (SSE_t) \right\}_{j=1}^J$$

t -th iteration SSE

Calculate the gradient to find the direction to move to minimize the SSE

- The gradient vector, $\mathbf{g}_t$, is the gradient at the iteration t .
- Each element in the gradient is the partial first derivative of the SSE with respect to the parameters:

$$\mathbf{g}_t = \left\{ \frac{\partial}{\partial \theta_{t,j}} (SSE_t) \right\}_{j=1}^J$$

Total number of parameters to learn: $J = H(D + 1) + H + 1$

We need to calculate the derivative with respect to each β and α parameter!

- The gradient vector, $\mathbf{g}_t$, is the gradient at the iteration t .
- Each element in the gradient is the partial first derivative of the SSE with respect to the parameters:

$$\mathbf{g}_t = \left\{ \frac{\partial}{\partial \theta_{t,j}} (SSE_t) \right\}_{j=1}^J = \left\{ \left\{ \left\{ \frac{\partial}{\partial \beta_{t,d,k}} (SSE_t) \right\}_{d=0}^D \right\}_{k=1}^H, \left\{ \frac{\partial}{\partial \alpha_{t,k}} (SSE_t) \right\}_{k=0}^H \right\}$$

Specifically for our toy problem a single input, $D = 1$, and 2 hidden units, $H = 2$

- The gradient vector, $\mathbf{g}_t$, is the gradient at the iteration t .
- Each element in the gradient is the partial first derivative of the SSE with respect to the parameters:

$$\mathbf{g}_t = \left[\frac{\partial(SSE_t)}{\partial \beta_{t,0,1}}, \frac{\partial(SSE_t)}{\partial \beta_{t,1,1}}, \frac{\partial(SSE_t)}{\partial \beta_{t,0,2}}, \frac{\partial(SSE_t)}{\partial \beta_{t,1,2}}, \frac{\partial(SSE_t)}{\partial \alpha_{t,0}}, \frac{\partial(SSE_t)}{\partial \alpha_{t,1}}, \frac{\partial(SSE_t)}{\partial \alpha_{t,2}} \right]^T$$

Once the gradient vector is calculated, we need to decide “how far to walk” along that direction

- When we discussed logistic regression, we worked through the steps of the IRLS algorithm.
- IRLS uses the **Newton method** to perform the optimization.
- The direction and distance in the “full” Newton method is modified by the Hessian matrix, the matrix of second derivatives!
 - We denoted the Hessian matrix in the IRLS algorithm as **H**, note that this is **NOT** the matrix of hidden unit responses...

The full Newton method update step is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{H}_t^{-1} \mathbf{g}_t$$

- The gradient vector is a $J \times 1$ column vector.
- The Hessian matrix is a $J \times J$ matrix.

The full Newton method update step is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{H}_t^{-1} \mathbf{g}_t$$

- The gradient vector is a $J \times 1$ column vector.
- The Hessian matrix is a $J \times J$ matrix.

For our toy problem, the Hessian matrix is just 7×7 .

Consider a more realistic problem, with $D = 5$ and $H = 15$, the total number of unknowns would be $15(5 + 1) + 15 + 1 = 106$!

The Hessian matrix would be 106×106 .

The full Newton method update step is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{H}_t^{-1} \mathbf{g}_t$$

- The gradient vector is a $J \times 1$ column vector.
- The Hessian matrix is a $J \times J$ matrix.

Consider a moderate sized problem with $D = 10$ and $H = 20$...total number of unknowns would be $20(10 + 1) + 20 + 1 = 241$.

Hessian matrix would be 241×241 !

The full Newton method update step is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{H}_t^{-1} \mathbf{g}_t$$

- The gradient vector is a $J \times 1$ column vector.
- The Hessian matrix is a $J \times J$ matrix.

Consider a bigger sized problem with $D = 50$ and $H = 200$...total number of unknowns would be $200(50 + 1) + 200 + 1 = 10401$.

Hessian matrix would be 10401×10401 !

Each iteration would require evaluating the second derivatives and **inverting** the matrix!

$$\mathbf{H}_t(j, j') = \frac{\partial^2 (SSE_t)}{\partial \theta_{t,j} \partial \theta_{t,j'}}$$

The Hessian matrix associated with our toy problem

$$\begin{bmatrix}
 \frac{\partial^2 SSE_t}{\partial \beta_{t01}^2} & \frac{\partial^2 SSE_t}{\partial \beta_{t01} \partial \beta_{t11}} & \frac{\partial^2 SSE_t}{\partial \beta_{t01} \partial \beta_{t02}} & \frac{\partial^2 SSE_t}{\partial \beta_{t01} \partial \beta_{t12}} & \frac{\partial^2 SSE_t}{\partial \beta_{t01} \partial \alpha_{t0}} & \frac{\partial^2 SSE_t}{\partial \beta_{t01} \partial \alpha_{t1}} & \frac{\partial^2 SSE_t}{\partial \beta_{t01} \partial \alpha_{t2}} \\
 & \frac{\partial^2 SSE_t}{\partial \beta_{t11}^2} & \frac{\partial^2 SSE_t}{\partial \beta_{t11} \partial \beta_{t02}} & \frac{\partial^2 SSE_t}{\partial \beta_{t11} \partial \beta_{t12}} & \frac{\partial^2 SSE_t}{\partial \beta_{t11} \partial \alpha_{t0}} & \frac{\partial^2 SSE_t}{\partial \beta_{t11} \partial \alpha_{t1}} & \frac{\partial^2 SSE_t}{\partial \beta_{t11} \partial \alpha_{t2}} \\
 & & \frac{\partial^2 SSE_t}{\partial \beta_{t02}^2} & \frac{\partial^2 SSE_t}{\partial \beta_{t02} \partial \beta_{t12}} & \frac{\partial^2 SSE_t}{\partial \beta_{t02} \partial \alpha_{t0}} & \frac{\partial^2 SSE_t}{\partial \beta_{t02} \partial \alpha_{t1}} & \frac{\partial^2 SSE_t}{\partial \beta_{t02} \partial \alpha_{t2}} \\
 & & & \frac{\partial^2 SSE_t}{\partial \beta_{t12}^2} & \frac{\partial^2 SSE_t}{\partial \beta_{t12} \partial \alpha_{t0}} & \frac{\partial^2 SSE_t}{\partial \beta_{t12} \partial \alpha_{t1}} & \frac{\partial^2 SSE_t}{\partial \beta_{t12} \partial \alpha_{t2}} \\
 & & & & \frac{\partial^2 SSE_t}{\partial \alpha_{t0}^2} & \frac{\partial^2 SSE_t}{\partial \alpha_{t0} \partial \alpha_{t1}} & \frac{\partial^2 SSE_t}{\partial \alpha_{t0} \partial \alpha_{t2}} \\
 & & & & & \frac{\partial^2 SSE_t}{\partial \alpha_{t1}^2} & \frac{\partial^2 SSE_t}{\partial \alpha_{t1} \partial \alpha_{t2}} \\
 & & & & & & \frac{\partial^2 SSE_t}{\partial \alpha_{t2}^2}
 \end{bmatrix}$$

To reduce the computational cost of the algorithm, `optim()` does not actually evaluate the Hessian matrix

- In our calls to `optim()`, we have set the `method` argument to "BFGS".
- The BFGS algorithm is a *quasi-Newton* algorithm which *approximates* the Hessian matrix *based on the gradient vector*.

To reduce the computational cost of the algorithm, `optim()` does not actually evaluate the Hessian matrix

- Denote the approximate Hessian matrix as: $\tilde{\mathbf{H}}_t \approx \mathbf{H}_t$
- The update step is then:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \tilde{\mathbf{H}}_t^{-1} \mathbf{g}_t$$

- BFGS performs this operation in an efficient and computationally stable way.

That said, quasi-Newton algorithms still require a lot of computations to complete a single iteration

- We need to approximate the inverse of the Hessian matrix!
- Instead, we could consider a simpler approach with gradient descent:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \mathbf{g}_t$$

- Where γ is the step size or learning rate.

Comparisons between gradient descent and quasi-Newton methods

- Gradient descent requires fewer **calculations** per iteration than a quasi-Newton method.
- Gradient descent however will require more **iterations** to find the optimum compared to a quasi-Newton method.
- Our simple implementation with `optim()` fit the neural network with about **30 iterations** while `neuralnet` **required over 200** for our simple toy problem.

Comparisons between gradient descent and quasi-Newton methods

- Gradient descent requires fewer **calculations** per iteration than a quasi-Newton method.
- Gradient descent however will require more **iterations** to find the optimum compared to a quasi-Newton method.
- **Trade-off exists between computational cost per iteration vs number of iterations.**

Gradient descent based approaches have become more popular compared to quasi-Newton methods for deep neural networks

- We will discuss why that is later in the presentation.

How does `optim()` actually calculate the gradient?

- In lecture and your assignments, the `gr` argument to `optim()` has been set to `NULL`.
- This tells `optim()` to estimate the gradient with **finite differences**.

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a forward difference:

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\theta_j}$$

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a forward difference:

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\theta_j}$$


Denotes all parameters EXCEPT the j -th parameter: $\boldsymbol{\theta}_{(t,-j)} = \theta_{t,j=1}, \theta_{t,j=2}, \dots, \theta_{t,j-1}, \theta_{t,j+1}, \dots, \theta_{t,j=J}$

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a forward difference:

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\theta_j}$$


All parameters including the j -th parameter: $\boldsymbol{\theta}_t = \theta_{t,j=1}, \theta_{t,j=2}, \dots, \theta_{t,j-1}, \theta_{t,j}, \theta_{t,j+1}, \dots, \theta_{t,j=J}$

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a forward difference:

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - \boxed{SSE_t(\boldsymbol{\theta}_t)}}{\Delta\theta_j}$$



The t -th iteration's
guess for the SSE

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a forward difference:

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\theta_j}$$


The change in the guess due to changing
1 and only 1 parameter

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a forward difference:

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\theta_j}$$

We must perturb each parameter individually!!!

Finite differences approximate derivatives by perturbation

- We need to define a perturbation, or small change, for each parameter: $\Delta\theta_j$
- We can approximate the first derivative of the SSE_t with respect to $\theta_{t,j}$ via a formula
We must recalculate the SSE_t at the perturbed parameter value, for each parameter!!

$$\frac{\partial}{\partial \theta_{t,j}} (SSE_t) \approx \frac{SSE_t(\boldsymbol{\theta}_{(t,-j)}, \theta_{t,j} + \Delta\theta_j) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\theta_j}$$

We must perturb each parameter individually!!!

For our simple toy problem, we have 4 hidden unit parameters and 3 output layer parameters

- The t -th iteration's set of parameter values:

$$\boldsymbol{\theta}_t = \{\beta_{t,0,1}, \beta_{t,1,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2}\}$$

- If the j -th parameter of interest is $\beta_{t,1,1}$ the set of all EXCEPT j are:

$$\boldsymbol{\theta}_{(t,-j)} = \{\beta_{t,0,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2}\}$$

For our simple toy problem, we have 4 hidden unit parameters and 3 output layer parameters. The finite difference approximation to the gradient:

$$\mathbf{g}_t = \begin{bmatrix} \frac{\partial(SSE_t)}{\partial \beta_{t,0,1}} \\ \frac{\partial(SSE_t)}{\partial \beta_{t,1,1}} \\ \frac{\partial(SSE_t)}{\partial \beta_{t,0,2}} \\ \frac{\partial(SSE_t)}{\partial \beta_{t,1,2}} \\ \frac{\partial(SSE_t)}{\partial \alpha_{t,0}} \\ \frac{\partial(SSE_t)}{\partial \alpha_{t,1}} \\ \frac{\partial(SSE_t)}{\partial \alpha_{t,2}} \end{bmatrix} \approx \begin{bmatrix} \frac{SSE_t(\beta_{t,0,1} + \Delta\beta_{0,1}, \beta_{t,1,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2}) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\beta_{0,1}} \\ \frac{SSE_t(\beta_{t,0,1}, \beta_{t,1,1} + \Delta\beta_{1,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2}) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\beta_{1,1}} \\ \frac{SSE_t(\beta_{t,0,1}, \beta_{t,1,1}, \beta_{t,0,2} + \Delta\beta_{0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2}) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\beta_{0,2}} \\ \frac{SSE_t(\beta_{t,0,1}, \beta_{t,1,1}, \beta_{t,0,2}, \beta_{t,1,2} + \Delta\beta_{1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2}) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\beta_{1,2}} \\ \frac{SSE_t(\beta_{t,0,1}, \beta_{t,1,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0} + \Delta\alpha_0, \alpha_{t,1}, \alpha_{t,2}) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\alpha_0} \\ \frac{SSE_t(\beta_{t,0,1}, \beta_{t,1,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1} + \Delta\alpha_1, \alpha_{t,2}) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\alpha_1} \\ \frac{SSE_t(\beta_{t,0,1}, \beta_{t,1,1}, \beta_{t,0,2}, \beta_{t,1,2}, \alpha_{t,0}, \alpha_{t,1}, \alpha_{t,2} + \Delta\alpha_2) - SSE_t(\boldsymbol{\theta}_t)}{\Delta\alpha_2} \end{bmatrix}$$

A finite difference based approach will not efficiently scale to large problems

- Just as we did with linear and generalized linear models, let's derive analytic expressions for the gradient!

Start by rewriting the SSE relative to each observation

- The following slides drop the iteration index, t , for simplicity.

$$SSE = R(\boldsymbol{\theta}) = \sum_{n=1}^N (R_n(\boldsymbol{\theta}))$$

- The n -th observation's contribution to the SSE is:

$$R_n(\boldsymbol{\theta}) = (y_n - f_n)^2$$

We need to calculate the derivatives of R_n with respect to each parameter

- Remember that there are two groups of parameters:
 - The hidden unit parameters – the betas.
 - The output layer parameters – the alphas.

$$\boldsymbol{\theta} = \{\boldsymbol{\beta}_1, \boldsymbol{\beta}_2, \dots, \boldsymbol{\beta}_k, \dots, \boldsymbol{\beta}_H, \alpha_0, \boldsymbol{\alpha}\}$$

- There are $H + 1$ alphas, and $D + 1$ betas for each hidden unit:

$$\boldsymbol{\theta} = \left\{ \left\{ \{\beta_{dk}\}_{d=0}^D \right\}_{k=1}^H, \{\alpha_k\}_{k=0}^H \right\}$$

Start with the output layer parameters

$$\frac{\partial R_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_k}$$

Start with the output layer parameters

$$\frac{\partial R_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_k}$$

- The neural network response, f_n , is a linear model relative to the hidden units → a linear basis model!

$$f_n = \sum_{k'}^H (\alpha_{k'} h_{n,k'}) = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2} + \cdots + \alpha_{k'} h_{n,k'} + \cdots + \alpha_H h_{n,H}$$

Start with the output layer parameters

$$\frac{\partial R_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_k}$$

- The derivative of f_n with respect to α_k :

$$\frac{\partial f_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} \left(\sum_{k'}^H (\alpha_{k'} h_{n,k'}) \right) = \sum_{k'}^H \left(\frac{\partial}{\partial \alpha_k} (\alpha_{k'} h_{n,k'}) \right)$$

Start with the output layer parameters

$$\frac{\partial R_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_k}$$

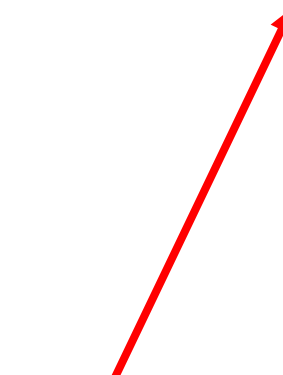
- The derivative of f_n with respect to α_k :

$$\frac{\partial f_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} \left(\sum_{k'=0}^H (\alpha_{k'} h_{n,k'}) \right) = h_{n,k}$$

Start with the output layer parameters

$$\frac{\partial R_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_k}$$

- The derivative of f_n with respect to α_k :

$$\frac{\partial f_n}{\partial \alpha_k} = \frac{\partial}{\partial \alpha_k} \left(\sum_{k=0}^H (\alpha_k h_{n,k}) \right) = h_{n,k}$$


Start with the output layer parameters

$$\frac{\partial R_n}{\partial \alpha_k} = -2(y_n - f_n)h_{n,k}$$

Let's see that expression again, but specifically for our simple toy problem

- We used two hidden units, $H = 2$.
- The neural network response in the output layer is just:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

Start with the output layer intercept or bias

- We used two hidden units, $H = 2$.
- The neural network response in the output layer is just:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

- Derivative of the n -th squared residual with respect to α_0 :

$$\frac{\partial R_n}{\partial \alpha_0} = \frac{\partial}{\partial \alpha_0} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_0}$$

Start with the output layer intercept or bias

- We used two hidden units, $H = 2$.
- The neural network response in the output layer is just:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

- Derivative of the neural network response with respect to α_0 :

$$\frac{\partial f_n}{\partial \alpha_0} = \frac{\partial}{\partial \alpha_0} (\alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}) = 1$$

Now the slope or weight acting on hidden unit 1

- We used two hidden units, $H = 2$.
- The neural network response in the output layer is just:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

- Derivative of the n -th squared residual with respect to α_1 :

$$\frac{\partial R_n}{\partial \alpha_1} = \frac{\partial}{\partial \alpha_1} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_1}$$

Now the slope or weight acting on hidden unit 1

- We used two hidden units, $H = 2$.
- The neural network response in the output layer is just:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

- Derivative of the neural network response with respect to α_1 :

$$\frac{\partial f_n}{\partial \alpha_1} = \frac{\partial}{\partial \alpha_1} (\alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}) = h_{n,1}$$

Derivative for hidden unit 2's weight is similar

- We used two hidden units, $H = 2$.
- The neural network response in the output layer is just:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

- Derivative of the neural network response with respect to α_2 :

$$\frac{\partial f_n}{\partial \alpha_2} = \frac{\partial}{\partial \alpha_2} (\alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}) = h_{n,2}$$

Output layer derivatives

$$\left[\frac{\partial f_n}{\partial \alpha_0}, \frac{\partial f_n}{\partial \alpha_1}, \frac{\partial f_n}{\partial \alpha_2} \right] = [1, h_{n,1}, h_{n,2}]$$

Output layer derivatives

$$\left[\frac{\partial f_n}{\partial \alpha_0}, \frac{\partial f_n}{\partial \alpha_1}, \frac{\partial f_n}{\partial \alpha_2} \right] = [1, h_{n,1}, h_{n,2}]$$

- Define a “fake” hidden unit or zeroth hidden unit: $h_{n,k=0} = 1$.
- The derivatives become:

$$\left[\frac{\partial f_n}{\partial \alpha_0}, \frac{\partial f_n}{\partial \alpha_1}, \frac{\partial f_n}{\partial \alpha_2} \right] = [h_{n,0}, h_{n,1}, h_{n,2}]$$

The R_n derivatives with respect to output layer parameters for the toy problem

$$\left[\frac{\partial f_n}{\partial \alpha_0}, \frac{\partial f_n}{\partial \alpha_1}, \frac{\partial f_n}{\partial \alpha_2} \right] = [h_{n,0}, h_{n,1}, h_{n,2}]$$

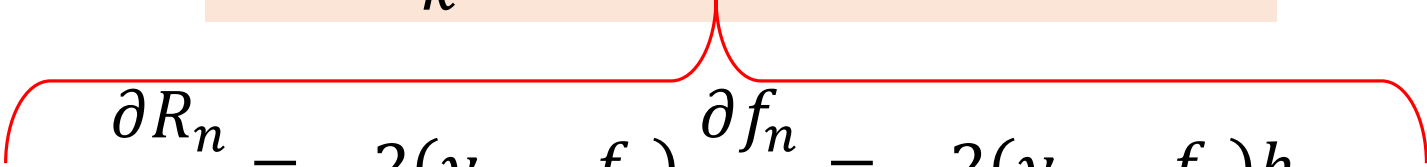
$$\frac{\partial R_n}{\partial \alpha_0} = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_0} = -2(y_n - f_n)h_{n,0}$$

$$\frac{\partial R_n}{\partial \alpha_1} = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_1} = -2(y_n - f_n)h_{n,1}$$

$$\frac{\partial R_n}{\partial \alpha_2} = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_2} = -2(y_n - f_n)h_{n,2}$$

In general for the k -th hidden unit:

$$\frac{\partial R_n}{\partial \alpha_k} = -2(y_n - f_n)h_{n,k}$$


$$\frac{\partial R_n}{\partial \alpha_0} = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_0} = -2(y_n - f_n)h_{n,0}$$

$$\frac{\partial R_n}{\partial \alpha_1} = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_1} = -2(y_n - f_n)h_{n,1}$$

$$\frac{\partial R_n}{\partial \alpha_2} = -2(y_n - f_n) \frac{\partial f_n}{\partial \alpha_2} = -2(y_n - f_n)h_{n,2}$$

Now consider the derivatives with respect to the hidden unit parameters

$$\frac{\partial R_n}{\partial \beta_{d,k}} = \frac{\partial}{\partial \beta_{d,k}} ((y_n - f_n)^2) = -2(y_n - f_n) \frac{\partial f_n}{\partial \beta_{d,k}}$$

- How does the neural net response, f_n , relate to the hidden unit parameter, $\beta_{d,k}$?

$$\frac{\partial f_n}{\partial \beta_{d,k}} = ?$$

Now consider the derivatives with respect to the hidden unit parameters

$$\frac{\partial R_n}{\partial \beta_{d,k}} = \frac{\partial}{\partial \beta_{d,k}} \left((y_n - f_n)^2 \right) = -2(y_n - f_n) \frac{\partial f_n}{\partial \beta_{d,k}}$$

- How does the neural net response, f_n , relate to the hidden unit parameter, $\beta_{d,k}$? **Use the chain rule!!**

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \frac{\partial h_{n,k}}{\partial \eta_{n,k}} \times \frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}$$

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \boxed{\frac{\partial f_n}{\partial h_{n,k}}} \times \frac{\partial h_{n,k}}{\partial \eta_{n,k}} \times \frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}$$

$$\frac{\partial f_n}{\partial h_{n,k}} = \frac{\partial}{\partial h_{n,k}} \left(\sum_{k'=0}^H (\alpha_{k'} h_{n,k'}) \right) = \alpha_k$$

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \boxed{\frac{\partial h_{n,k}}{\partial \eta_{n,k}}} \times \frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}$$

- Depends on the non-linear “activation” function!
- For generality, denote the derivative as:

$$\frac{\partial h_{n,k}}{\partial \eta_{n,k}} = \frac{\partial}{\partial \eta_{n,k}} (h_{n,k}) = \frac{\partial}{\partial \eta_{n,k}} (g(\eta_{n,k})) = g'_{n,k}$$

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \boxed{\frac{\partial h_{n,k}}{\partial \eta_{n,k}}} \times \frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}$$

- Depends on the non-linear “activation” function!
- For generality, denote the derivative as:

$$\frac{\partial h_{n,k}}{\partial \eta_{n,k}} = \frac{\partial}{\partial \eta_{n,k}} (h_{n,k}) = \frac{\partial}{\partial \eta_{n,k}} (g(\eta_{n,k})) = g'_{n,k}$$

- If we are using the logistic function as the “activation” function:

$$g'_{n,k} = h_{n,k}(1 - h_{n,k})$$

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \frac{\partial h_{n,k}}{\partial \eta_{n,k}} \times \boxed{\frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}}$$

$$\frac{\partial \eta_{n,k}}{\partial \beta_{dk}} = \frac{\partial}{\partial \beta_{dk}} \left(\sum_{d'=0}^D (\beta_{d'k} x_{n,d'}) \right) = x_{n,d}$$

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \frac{\partial h_{n,k}}{\partial \eta_{n,k}} \times \boxed{\frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}}$$

$$\frac{\partial \eta_{n,k}}{\partial \beta_{dk}} = \frac{\partial}{\partial \beta_{dk}} \left(\sum_{d'=0}^D (\beta_{d'k} x_{n,d'}) \right) = x_{n,d}$$

This is just the derivative of the generalized linear model!!!

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \frac{\partial h_{n,k}}{\partial \eta_{n,k}} \times \boxed{\frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}}$$

$$\frac{\partial \eta_{n,k}}{\partial \beta_{dk}} = \frac{\partial}{\partial \beta_{dk}} \left(\sum_{d'=0}^D (\beta_{d'k} x_{n,d'}) \right) = x_{n,d}$$

Derivative of the linear predictor, η , with respect to the d -th weight, β_d , is equal to the d -th input, x_d !!!

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \frac{\partial f_n}{\partial h_{n,k}} \times \frac{\partial h_{n,k}}{\partial \eta_{n,k}} \times \boxed{\frac{\partial \eta_{n,k}}{\partial \beta_{d,k}}}$$

$$\frac{\partial \eta_{n,\boxed{k}}}{\partial \beta_{d,\boxed{k}}} = \frac{\partial}{\partial \beta_{dk}} \left(\sum_{d'=0}^D (\beta_{d'k} x_{n,d'}) \right) = x_{n,d}$$

However, with neural networks we have to do this for all hidden units: $k = 1, \dots, H$

Combine the terms together

$$\frac{\partial f_n}{\partial \beta_{d,k}} = \alpha_k \times g'_{n,k} \times x_{n,d}$$

- The derivative of R_n with respect to $\beta_{d,k}$ is then:

$$\frac{\partial R_n}{\partial \beta_{d,k}} = -2(y_n - f_n) \frac{\partial f_n}{\partial \beta_{d,k}} = -2(y_n - f_n) \alpha_k g'_{n,k} x_{n,d}$$

Let's walk through those steps for the simple toy problem

- The output layer for the n -th observation, is a linear model relative to the hidden units:

$$f_n = \alpha_0 + \alpha_1 h_{n,1} + \alpha_2 h_{n,2}$$

- The hidden units are non-linear transformations of the linear predictors:

$$f_n = \alpha_0 + \alpha_1 g(\eta_{n,1}) + \alpha_2 g(\eta_{n,2})$$

- The linear predictors are just linear models relative to the inputs!

$$f_n = \alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1})$$

If we want to calculate the derivative of f_n with respect to β_{11}

$$f_n = \alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1})$$

Hidden unit 1 depends
on β_{11}

Hidden unit 2 does
NOT depend on β_{11}

“Build” back to the hidden unit response $h_{n,1}$

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_1 g(\underbrace{\beta_{0,1} + \beta_{1,1} x_{n,1}}_{\eta_{n,1}}) \right)$$

“Build” back to the hidden unit response $h_{n,1}$

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_1 g(\eta_{n,1}) \right)$$

“Build” back to the hidden unit response $h_{n,1}$

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_1 \underbrace{g(\eta_{n,1})}_{h_{n,1}} \right)$$

“Build” back to the hidden unit response $h_{n,1}$

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} (\alpha_1 h_{n,1})$$

Work through the derivatives

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



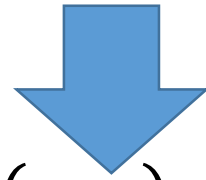
$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} (\alpha_1 h_{n,1}) = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}}$$

Work through the derivatives

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} (\alpha_1 h_{n,1}) = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}}$$



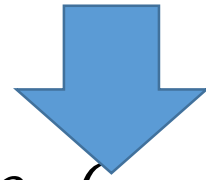
$$\frac{\partial f_n}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial g(\eta_{n,1})}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial g(\eta_{n,1})}{\partial \eta_{n,1}} \times \frac{\partial \eta_{n,1}}{\partial \beta_{1,1}}$$

Define $g'_{n,1}$ as the derivative of the transformation

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} (\alpha_1 h_{n,1}) = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}}$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial g(\eta_{n,1})}{\partial \beta_{1,1}} = \alpha_1 g'_{n,1} \times \frac{\partial \eta_{n,1}}{\partial \beta_{1,1}}$$

Finally, take the derivative of the linear predictor

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} (\alpha_1 h_{n,1}) = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}}$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}} = \alpha_1 g'_{n,1} \times \frac{\partial}{\partial \beta_{1,1}} (\beta_{0,1} + \beta_{1,1} x_{n,1})$$

We get what we had before, just specifically for the $\beta_{1,1}$

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} (\alpha_1 h_{n,1}) = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}}$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \alpha_1 \frac{\partial h_{n,1}}{\partial \beta_{1,1}} = \alpha_1 g'_{n,1} \times x_{n,1}$$

We get what we had before, just specifically for the β_{11}

$$\frac{\partial f_n}{\partial \beta_{1,1}} = \frac{\partial}{\partial \beta_{1,1}} \left(\alpha_0 + \alpha_1 g(\beta_{0,1} + \beta_{1,1} x_{n,1}) + \alpha_2 g(\beta_{0,2} + \beta_{1,2} x_{n,1}) \right)$$



$$\frac{\partial f_n}{\partial \beta_{1,1}} = \alpha_1 g'_{n,1} x_{n,1}$$

Back to the general notation, the two groups of derivatives are:

$$\frac{\partial R_n}{\partial \alpha_k} = -2 \times (y_n - f_n) \times h_{n,k}$$

$$\frac{\partial R_n}{\partial \beta_{dk}} = -2 \times (y_n - f_n) \times \alpha_k g'_{n,k} x_{n,k}$$

The two groups of derivatives share something in common...

$$\frac{\partial R_n}{\partial \alpha_k} = -2 \times (y_n - f_n) \times h_{n,k}$$

$$\frac{\partial R_n}{\partial \beta_{dk}} = -2 \times (y_n - f_n) \times \alpha_k g'_{n,k} x_{n,k}$$

The two groups of derivatives share something in common...

$$\frac{\partial R_n}{\partial \alpha_k} = -2 \times (y_n - f_n) \times h_{n,k}$$

The error: $\delta_n = y_n - f_n$

$$\frac{\partial R_n}{\partial \beta_{dk}} = -2 \times (y_n - f_n) \times \alpha_k g'_{n,k} x_{n,k}$$

Rewrite the derivatives in terms of the error or the delta:

$$\frac{\partial R_n}{\partial \alpha_k} = -2\delta_n h_{n,k}$$

$$\frac{\partial R_n}{\partial \beta_{dk}} = -2\delta_n \alpha_k g'_{n,k} x_{n,d}$$

Define the hidden unit “error” in terms of the output layer delta:

$$\xi_{n,k} = \alpha_k g'_{n,k} \delta_n$$

- The derivative with respect to the hidden unit parameter is then:

$$\frac{\partial R_n}{\partial \beta_{dk}} = -2\xi_{n,k} x_{n,d}$$

Key expressions for calculating the derivatives:

- Error: $\delta_n = y_n - f_n$
- Output layer parameter derivative: $\frac{\partial R_n}{\partial \alpha_k} = -2\delta_n h_{n,k}$
- Hidden unit “error”: $\xi_{n,k} = \alpha_k g'_{n,k} \delta_n$
- Hidden unit parameter derivative: $\frac{\partial R_n}{\partial \beta_{dk}} = -2\xi_{n,k} x_{n,d}$

With these expressions, we can backpropagate the error from the **output layer** to the **hidden units**!

The backpropagation method is:

- Given input design matrix, $\mathbf{X}$, and current guess for $\boldsymbol{\theta}$:
 - Calculate all hidden units, $\mathbf{H}$
 - Calculate the neural network response, $\mathbf{f}$
- Given $\mathbf{y}$ and $\mathbf{f}$, calculate the deltas, $\boldsymbol{\delta}$
- Backpropagate $\{\delta_n\}_{n=1}^N$ to calculate all $\left\{ \left\{ \xi_{n,k} \right\}_{k=1}^H \right\}_{n=1}^N$
- Calculate all derivatives

With these expressions, we can backpropagate the error from the **output layer** to the **hidden units**!

The backpropagation method is:

- Given input design matrix, $\mathbf{X}$, and current guess for $\boldsymbol{\theta}$:
 - Calculate all hidden units, $\mathbf{H}$
 - Calculate the neural network response, $\mathbf{f}$
- Given $\mathbf{y}$ and $\mathbf{f}$, calculate the deltas, $\boldsymbol{\delta}$
- Backpropagate $\{\delta_n\}_{n=1}^N$ to calculate all $\left\{\left\{\xi_{n,k}\right\}_{k=1}^H\right\}_{n=1}^N$
- Calculate all derivatives

Forward step

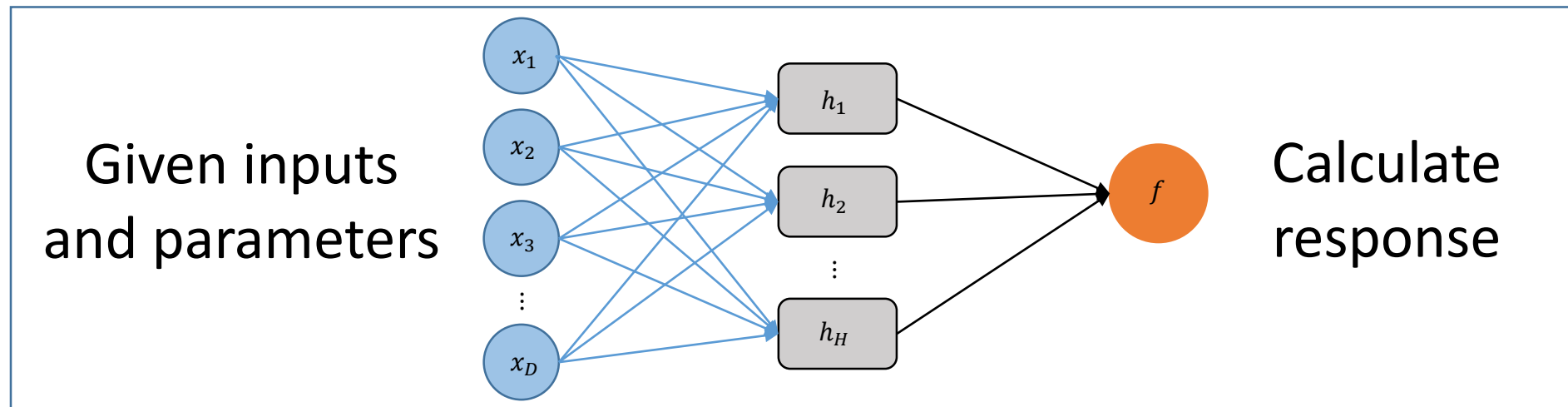


Propagate
error
backwards

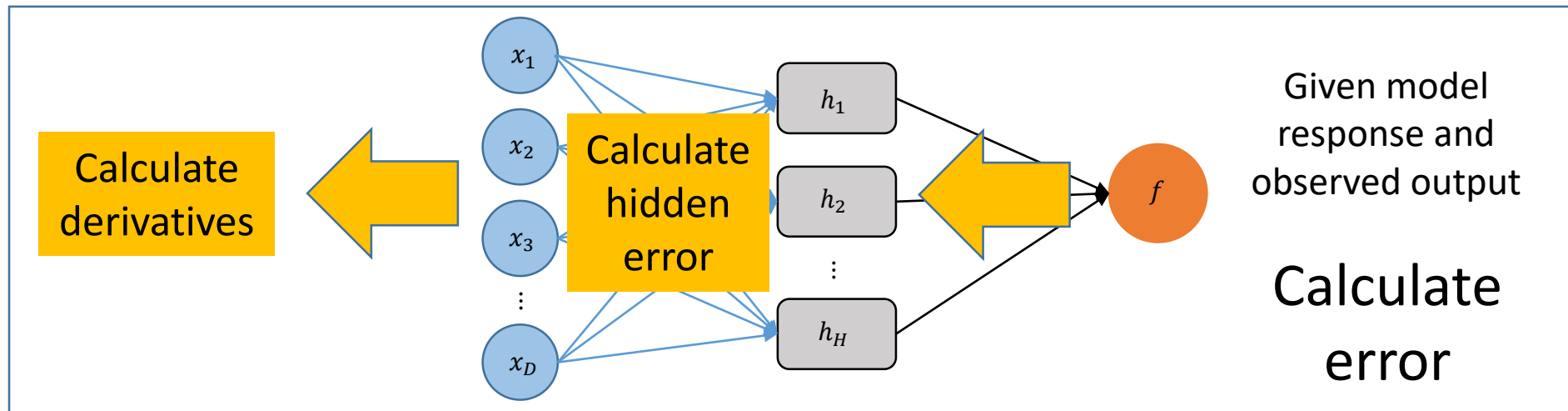
With these expressions, we can backpropagate the error from the **output layer** to the **hidden units**!

The forward step is:

- Given input design matrix, $\mathbf{X}$, and current guess for θ :
 - Calculate all hidden units, $\mathbf{H}$
 - Calculate the neural network response, $\mathbf{f}$



With these expressions, we can backpropagate the error from the **output layer** to the **hidden units**!



- Given $\mathbf{y}$ and $\mathbf{f}$, calculate the deltas, δ
- Backpropagate $\{\delta_n\}_{n=1}^N$ to calculate all $\left\{ \left\{ \xi_{n,k} \right\}_{k=1}^H \right\}_{n=1}^N$
- Calculate all derivatives

General iterative optimization scheme

- Given $\mathbf{X}$, $\mathbf{y}$ (the training data)
- Specify starting guess, $\boldsymbol{\theta}_{t=0}$
- **Backpropagation**
 - Forward step
 - Propagate errors backwards
 - **Calculate derivatives**
- Update the parameters, $\boldsymbol{\theta}_{t+1}$
- Repeat until gradient near zero

General iterative optimization scheme

- Given $\mathbf{X}$, $\mathbf{y}$ (the training data)
- Specify starting guess, $\theta_{t=0}$
- **Backpropagation**
 - Forward step
 - Propagate errors backwards
 - **Calculate derivatives**
- Update the parameters, θ_{t+1}
- Repeat until gradient near zero



Can use gradient descent or
quasi-Newton algorithms like
BFGS

We wrote out the derivatives in terms of the n -th squared error

- The “complete” derivatives require summing over **ALL** observations.
- For the general parameter θ_j :

$$\frac{\partial R}{\partial \theta_j} = \sum_{n=1}^N \left(\frac{\partial R_n}{\partial \theta_j} \right)$$

We wrote out the derivatives in terms of the n -th squared error

- The “complete” derivatives require summing over **ALL** observations.
- For the individual output layer and hidden unit parameters:

$$\frac{\partial R}{\partial \alpha_k} = \sum_{n=1}^N \left(\frac{\partial R_n}{\partial \alpha_k} \right)$$

$$\frac{\partial R}{\partial \beta_{d,k}} = \sum_{n=1}^N \left(\frac{\partial R_n}{\partial \beta_{d,k}} \right)$$

Batch mode – update the parameters based on ALL training points

- Calculate the “complete” derivatives before updating the parameters:

$$\frac{\partial R}{\partial \alpha_k} = \sum_{n=1}^N \left(\frac{\partial R_n}{\partial \alpha_k} \right), \frac{\partial R}{\partial \beta_{d,k}} = \sum_{n=1}^N \left(\frac{\partial R_n}{\partial \beta_{d,k}} \right)$$

- One complete “sweep” is referred to as a **training epoch**.
- Batch mode updates the parameters at the end of an epoch.
- IMPORTANT: the derivatives associated with each observation can be calculated in parallel!

If the training set is large, for example $N > 10^5$, we have to perform a lot of calculations!

- Perhaps not all of the observations are equally “important”.
- For example, if several observations have nearly the same input values, $\mathbf{x}_{n,:} \approx \mathbf{x}_{n',:}$, the derivatives associated with those two observations will be similar.

Select a subset of observations to approximate the gradient

- Rather than completing an entire sweep, or a complete epoch, before updating the parameters, approximate the complete derivative by a subset of the observations.
- Choose M observations out the N total observations, $M \ll N$

$$\frac{\partial R}{\partial \alpha_k} \approx \sum_{m=1}^M \left(\frac{\partial R_m}{\partial \alpha_k} \right), \frac{\partial R}{\partial \beta_{d,k}} \approx \sum_{m=1}^M \left(\frac{\partial R_m}{\partial \beta_{d,k}} \right)$$

Select a subset of observations to approximate the gradient

$$\mathbf{g}_t \approx \tilde{\mathbf{g}}_t = \left\{ \left\{ \sum_{m=1}^M \left(\frac{\partial R_{t,m}}{\partial \beta_{t,d,k}} \right) \right\}_{d=0}^D \right\}_{k=1}^H, \left\{ \sum_{m=1}^M \left(\frac{\partial R_{t,m}}{\partial \alpha_{t,k}} \right) \right\}_{k=0}^H \right\}$$

- Update the parameters based on the approximate gradient with gradient descent:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \tilde{\mathbf{g}}_t$$

M can be 1 or some fraction of the total observations

- If $M = 1$ and the observation is chosen randomly the algorithm is referred as **stochastic gradient descent**.
- If $1 < M < N$ the algorithm is usually referred to as **mini-batch training**.

As useful as stochastic gradient descent (SGD) is, more efficient algorithms have been developed

- The more efficient approaches deal with better estimations of the learning rate in the update step:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \tilde{\mathbf{g}}_t$$

- Remember that the Newton method uses the curvature to tell us how far to walk.
- SGD requires setting that step size up front, so a poor choice means the algorithm could be VERY slow.

As useful as stochastic gradient descent (SGD) is, more efficient algorithms have been developed

- The more efficient approaches deal with better estimations of the learning rate in the update step:

$$\theta_{t+1} = \theta_t - \gamma \tilde{\mathbf{g}}_t$$

- More efficient approaches allow the learning rate to adapt without having to estimate the Hessian matrix, like a quasi-Newton algorithm.
- Approaches use the “momentum” to allow the learning rate to adapt.
- The [Adam optimizer](#) is based on this idea.

We have focused on the output layer parameters and the hidden unit parameters

- Remember that minimizing the SSE is analogous to maximizing the Gaussian log-likelihood.
- So what about the noise, σ ?

Conventional approach to estimate σ is just like the estimate from linear models.

- Find the optimal parameter estimates: $\hat{\theta}$
- Estimate the squared noise as the average squared error!

$$\hat{\sigma}^2 \approx \frac{SSE(\hat{\theta})}{N}$$

We spent a lot of time talking about complexity and overfitting

- We saw how linear and generalized linear models can overfit to the training data set.
- Using more and more terms drives up complexity, which increases the chance of overfitting.

Same holds true for neural networks!

- We can have a huge number of parameters.
- Neural networks can easily overfit to noise in the data set.

Early approach to overcome this was “early stopping”

- Start the parameters around 0 and stop the fitting algorithm before it reaches the optimal values.

However, we discussed how regularization can guard against overfitting.

- Regularization “pulls” parameters closer to zero.
- The same can be applied to neural networks!
- Neural network literature usually refers to regularization as **weight decay**.

Separate regularization factors are applied to the hidden units and output layer parameters

$$\sum_{n=1}^N (R_n(\boldsymbol{\theta})) + \lambda_{\beta} \sum_{k=1}^H \left(\sum_{d=1}^D (\beta_{d,k}^2) \right) + \lambda_{\alpha} \sum_{k=0}^H (\alpha_k^2)$$

Separate regularization factors are applied to the hidden units and output layer parameters

$$\sum_{n=1}^N (R_n(\boldsymbol{\theta})) + \lambda_{\beta} \sum_{k=1}^H \left(\sum_{d=1}^D (\beta_{d,k}^2) \right) + \lambda_{\alpha} \sum_{k=0}^H (\alpha_k^2)$$

What is this equivalent to????

Separate regularization factors are applied to the hidden units and output layer parameters

$$\sum_{n=1}^N (R_n(\boldsymbol{\theta})) + \lambda_{\beta} \sum_{k=1}^H \left(\sum_{d=1}^D (\beta_{d,k}^2) \right) + \lambda_{\alpha} \sum_{k=0}^H (\alpha_k^2)$$

Independent zero mean
Gaussian priors!